Introduction to Software Engineering

Project Proposal & Plan

The student team is required to complete the **Project Proposal & Plan** documentation for the assigned course project, following the attached template.



Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

Objectives 1
1 Member Contribution Assessment 2
2 Vision 3
    2.1 Preliminary Problem Statement 3
    2.2 Product Position Statement 5
    2.3 Stakeholders and Users 6
    2.4 Scope and Key Features 6
3 Proposed Solution 8
    3.1 Preliminary Architecture 8
    3.2 Technology Stack 9
    3.3 Hardware and Operating Environment 10
4 Project Plan 11
    4.1 Process and Phase Gates 11
    4.2 Work Breakdown Structure (WBS) 11
    4.3 Estimation 12
    4.4 Schedule 16
    4.5 Roles and Responsibilities 17
    4.6 Human Resources and Cost Estimate 18
5 Risk Management 20
6 Tools & Collaboration Setup 22
7 AI Usage Log 24

Project Proposal & Plan

Objectives

This document focus on the following topics:

- ✔ Completing the Project Proposal & Plan document with the following sections:
  - Vision
  - Proposed Solution
  - Project Plan
  - Risk Management
  - Tools & Collaboration Setup
- ✔ Including an AI usage log for this phase
- ✔ Understanding the Project Proposal & Plan document.

1 Member Contribution Assessment

ID	Name	Role(s)	Contribution (%)	Evidence
24127572	Nguyễn Cao Triết	Team Lead / PM; Developer	25%	SCRUM-6
24127447	Phùng Gia Mẫn	Business Analyst; Tester	25%	SCRUM-7
24127323	Tống Vân Anh	Designer / Architect; Developer	25%	SCRUM-8
24127207	Nguyễn Thục Nghi	Developer; Tester	25%	SCRUM-9

2 Vision

2.1 Preliminary Problem Statement

- **Business Description:**

As a software development team, we are required to develop a web-based applicant tracking system named miniTA. Currently, small companies often manage their recruitment processes through fragmented tools, resulting in scattered application data and potentially overlooked candidates. Furthermore, manually reviewing unstructured PDF resumes without predefined evaluation rubrics can lead to inconsistent skill assessments. Meanwhile, candidates lack visibility into their application status, forcing recruiters to spend valuable time responding to repetitive inquiries instead of focusing on candidate evaluation.

To resolve these issues, miniTA centralizes the recruitment workflow around each job posting. When initiating a recruitment campaign, recruiters create a job posting and define standardized evaluation criteria (rubrics), including required skills, preferred skills, experience requirements, evaluation weightings, and screening questions. The system also provides an AI-assisted job-intake feature that can draft evaluation criteria based on a pasted job description. However, recruiters must review and approve the suggested criteria before they can be used. Once the job posting is ready, the system provides a public application link that recruiters can share with potential candidates.

On the candidate side, applicants access the public link to submit their contact information, answer screening questions, and upload one PDF resume. After successfully submitting an application, candidates can receive a time-limited email magic link that allows them to securely access their submitted information and view their current application status. This feature reduces the need for candidates to contact the HR department for updates and allows recruiters to focus more on evaluating applications.

A core feature of miniTA is its automated resume processing and transparent matching mechanism. When a candidate uploads a PDF resume, the system extracts its selectable text and calculates a matching score against the job description and recruiter-approved evaluation rubric.

The system presents a breakdown of matched skills, missing requirements, and supporting text evidence extracted from the resume. These results help recruiters understand how each candidate's qualifications relate to the job requirements. Through the internal applicant inbox, recruiters can search, filter, and sort applications, add internal notes, complete human evaluation scorecards, and update candidate statuses. Additionally, a recruitment dashboard provides an overview of application counts, status distributions, matching-score distributions, and basic recruitment pipeline statistics.

To maintain a focused and feasible scope, miniTA is designed as an internal recruitment tracking workspace rather than a public job marketplace or social network. The system does not automatically reject, shortlist, or hire candidates based on matching scores. Instead, automated matching serves only as a decision-support mechanism, while all final recruitment decisions remain the responsibility of human reviewers.

- **Operating Environment:**

miniTA will be developed as a responsive web application accessible through modern web browsers that support HTML5 on desktop and mobile devices. Both recruiters and candidates will interact with the system through a web-based interface without requiring a dedicated desktop application. The application is intended to be deployed on a cloud server to support access to public application links and the internal recruitment workspace. Access to recruitment records and candidate information will be controlled according to user roles and authorization permissions.

- **Design & Implementation Constraints:**
    - **Frontend**: The user interface will be developed using ReactJS and TypeScript to support a responsive, interactive, and maintainable web application.
    - **Backend**: The server-side application will be implemented using Python and FastAPI to handle business logic, authentication, application processing, and communication with the database.
    - **Database**: PostgreSQL will serve as the primary relational database for storing user accounts, job postings, evaluation rubrics, candidate information, applications, matching results, and application statuses.

- **Data Processing:** PDF text extraction will use a suitable Python library, such as pdfplumber or PyMuPDF. pandas may be used for data manipulation, while a rule-based matching engine will compare extracted resume content against recruiter-approved evaluation criteria.
- **Scope Constraints**: The initial release will support PDF resumes containing selectable text only. OCR for scanned resumes, public job-board distribution, social networking, advanced calendar and video interview integrations, and automated hiring or rejection decisions will be excluded from the initial scope.
- **AI Assistance Constraints**: AI assistance through an LLM API will be limited to drafting evaluation rubrics and summaries. AI-generated outputs must be validated against the expected structured JSON format and reviewed by a recruiter before use. Manual alternatives must remain available when the AI service is unavailable or produces invalid output.
- **Data Privacy Constraints**: Because resumes contain sensitive personal information, the system must enforce appropriate authentication and authorization controls, obtain candidate consent where required, and use fictional or consented test data during development and testing. Access to uploaded resumes and candidate records must be restricted to authorized users.
-

2.2 Product Position Statement

Element	Content
For	Small companies and their HR/recruitment teams.
Who	Struggle with fragmented tools, inconsistent manual resume evaluations without predefined rubrics, and the time-consuming task of answering candidate status inquiries.
The (product name)	miniTA is a web-based Applicant Tracking System (ATS).
That	Centralizes recruitment workflows, automatically calculates explainable resume-matching scores with supporting text evidence, and enables applicants to securely track their status via an email magic link.
Unlike	Fragmented manual tools (spreadsheets, emails), complex enterprise platforms (e.g., SmartRecruiters, Workable), and public job boards (e.g., TopCV, VietnamWorks).

Our product	Acts strictly as an internal review aid rather than an automated decision-maker. It combines job posting, rubric-based matching, and secure candidate tracking in one lightweight platform, ensuring all final hiring or rejection decisions remain entirely in the hands of human reviewers.

2.3 Stakeholders and Users

#	Stakeholder / user	Description	Main interests
1	Hiring Managers	Department leaders who request new hires and oversee final recruitment decisions.	Need limited review access to evaluate shortlisted candidates efficiently using transparent, skills-based evidence.
2	Recruiters / HR	Internal HR staff responsible for managing the daily end-to-end recruitment pipeline.	Need to create job postings, configure evaluation rubrics, centralize applications, review resume matching evidence, and update candidate statuses.
3	Candidates	External job seekers applying for open positions at the company.	Need a simple public link to apply, upload a PDF resume, and securely track their application status via a time-limited email magic link.

2.4 Scope and Key Features

ID	Feature	Description	Priority (M/S/C/W)
F01	Account & Role Management	Recruiters log in and access specific records. Hiring managers get limited review access.	Must
F02	Job Posting Management	Create, edit, publish, close, and share job postings with public links.	Must
F03	Rubric Builder & AI Job-Intake Assistant	Recruiters define required/preferred skills, experience, weights, and screening questions. AI may draft these from a pasted JD, but the recruiter must approve them.	Must
F04	Candidate Application Portal	Candidates apply through the public link, provide contact information, answer screening questions, and upload one PDF CV/resume.	Must
F05	Candidate Status Portal	Candidates receive a time-limited email magic link to securely view their submitted application and current status.	Must

F06	Automated CV Extraction & Transparent Matching	System extracts text from PDF CVs, calculates a match score (%) against the JD/approved rubric, and shows a breakdown of matched/missing skills with text evidence.	Must
F07	Applicant Inbox & Human Review Workflow	Recruiters search, filter, sort, add notes, submit human scorecards, and update candidate statuses.	Must
F08	Recruitment Dashboard	Recruiters view application counts by job and status, score distribution, and basic pipeline conversion statistics.	Must
F09	Inbound Application Email	Candidates send a resume to a job-specific email address, and the system automatically creates an application from the attachment.	Won't
F10	Interview Scheduling & Email Templates	Integrated email notifications allowing recruiters to send reusable status emails or interview invitations directly from the system	Won't

Out of scope:

- ✔ **Automated Decisions:** The system will never make automatic hiring, shortlisting, or rejection decisions based on matching scores. It strictly acts as a decision-support review aid.
- ✔ **Optical Character Recognition (OCR):** The system will not support OCR for scanned or image-based resumes; it only processes selectable-text PDFs.
- ✔ **Public Marketplace Features:** Social networking, social feeds, job recommendations, and job-board distribution functionalities are excluded.
- ✔ **Advanced Integrations:** Calendar scheduling and video interview integrations are out of scope.

3 Proposed Solution

3.1 Preliminary Architecture

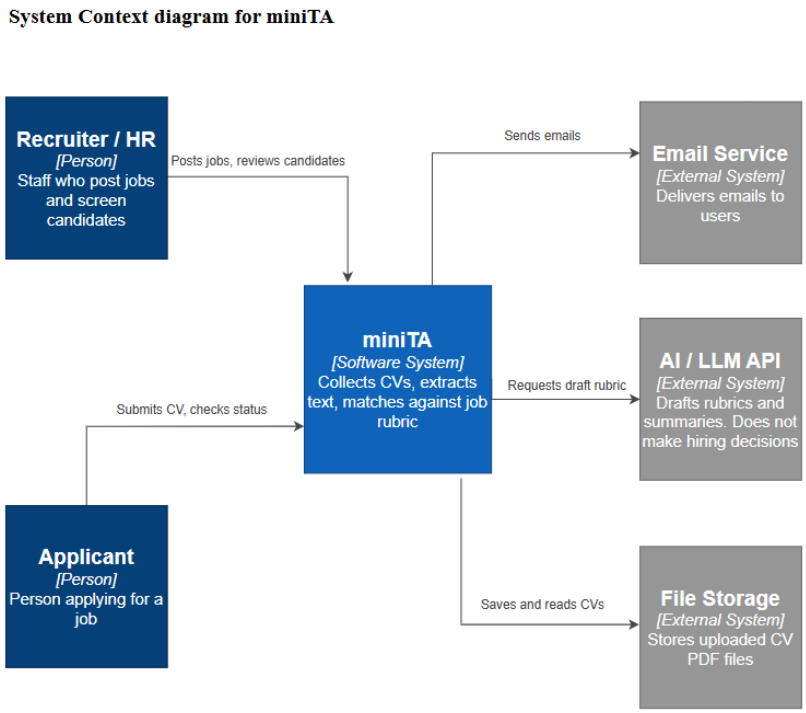


Figure 1. System Context diagram for miniTA

Figure 1 shows the System Context of miniTA. miniTA is a responsive web application used by two groups: Recruiters/HR, who create job postings and rubrics and review candidates, and Applicants, who submit a PDF CV and track their application status through an email magic link.

miniTA relies on three external systems. An Email Service delivers magic links and status emails. An AI/LLM API drafts rubric suggestions and summaries from a pasted job description; its output is validated and must be approved by the recruiter, so the AI never makes hiring decisions. A File Storage service keeps uploaded CV PDF files in a private location.

System boundary. Everything the team builds is inside miniTA: the web interface, API, authentication and roles, job and rubric management, PDF text extraction, matching engine, review workflow, dashboard and database. Email delivery, the LLM and file storage are outside the boundary; miniTA only calls them.

Storage of CV files and metadata. The original PDF files are stored in File Storage, not in PostgreSQL. PostgreSQL stores each file's metadata (file ID, original name, size, type, storage key, upload time, and the application it belongs to) together with the extracted text and match results. CV files are never publicly accessible; they are read only through requests that pass miniTA's authorization checks.

3.2 Technology Stack

Layer	Technology	Reason for choice
Front end	React + TypeScript (Vite as build tool)	A component-based UI is reused across the recruiter inbox, dashboard and applicant portal. TypeScript catches type errors early when four people edit the same code. The result is a responsive web app that works on desktop and mobile browsers without a separate mobile app.
Back end	Python + FastAPI; pdfplumber or PyMuPDF; pandas; modular monolith	Python has mature libraries for PDF text extraction and text processing, which are the core of the project. FastAPI validates request data automatically, and its schemas are also used to validate the structured JSON returned by the LLM. A modular monolith (one deployable application split into modules such as authentication, jobs, applications, extraction, matching and review) is simple to build, test and deploy for a team of four in three weeks.
Database	PostgreSQL	Jobs, rubrics, applications, statuses and scorecards are relational data that need constraints and transactions. JSON columns can hold rubrics and match breakdowns. The database stores file metadata and extracted text, not the PDF files themselves.

Hosting / deployment	Docker and Docker Compose; one cloud host or PaaS for the demo (provider to be confirmed in PA3)	Containers give every member the same environment and make deployment repeatable. A single deployable application keeps hosting effort and cost low.
Testing & CI	pytest (back end), Vitest (front end), GitHub Actions	Tests run automatically on every pull request, which enforces the Team Contract rule that a feature is complete only after it passes testing and review. GitHub Actions lives in the same repository as the code.

3.3 Hardware and Operating Environment

Development environment. Each member develops on a personal laptop (Windows, macOS or Linux) with a modern browser (Chrome or Edge), Git, VS Code, Node.js (LTS), Python 3.11 or newer, and Docker Desktop to run PostgreSQL locally. No special hardware is needed: the LLM is an external API, so no GPU or local model is required. Only fictional or consented test CVs are used.

Testing environment. Automated unit and integration tests run on GitHub-hosted Linux runners (GitHub Actions) on every pull request, with a temporary PostgreSQL database. System testing is done manually on a local Docker Compose setup or on the demo deployment, using Chrome and Edge on desktop and a phone-sized browser window for the applicant pages.

Deployment environment. For the demo, miniTA runs as containers on a single cloud host or PaaS (provider to be confirmed in PA3): the FastAPI back end, the React front end, a PostgreSQL database and a private file store for CV PDFs. A small instance is assumed to be enough for demo-scale traffic. The server needs HTTPS and outbound access to the email service and the LLM API, and secrets are provided through environment variables. End users only need a modern web browser and an internet connection.

4 Project Plan

4.1 Process and Phase Gates

The project follows the waterfall model. Each phase ends with a phase gate (a PA submission). After a PA is submitted it becomes a baseline; later changes must be updated in the document and listed in the next PA.

PA	Phase	Due	Exit criteria
PA1	Initiation & planning	End of week 3	Vision, scope, preliminary architecture, project plan, risk register, collaboration setup and AI usage log are reviewed; assigned Jira tasks provide contribution evidence.
PA2	Requirements	End of week 5	The SRS contains baselined functional, non-functional and domain requirements; Must-have use cases, mockups, acceptance criteria, a traceability matrix and a recorded requirements review are complete.
PA3	Design	End of week 7	Architecture, database, API, component and UI designs cover all baselined Must-have requirements; major decisions and requirement-to-design traceability are recorded.
PA4	Test planning	End of week 8	The test strategy, environment, test data and test cases are reviewed; every Must-have acceptance criterion maps to at least one test case.
PA5	Implementation, testing & deployment	End of week 10	Must-have features are implemented and deployed; automated and manual tests are executed; results, known defects, setup and deployment instructions are documented.
Final	Presentation & demo	Week 11	The deployed end-to-end recruitment flow is demonstrated; presentation materials are complete; every member can explain their contribution and answer questions.

4.2 Work Breakdown Structure (WBS)

Link WBS: SE06_PA1_WBS.xlsx

4.3 Estimation

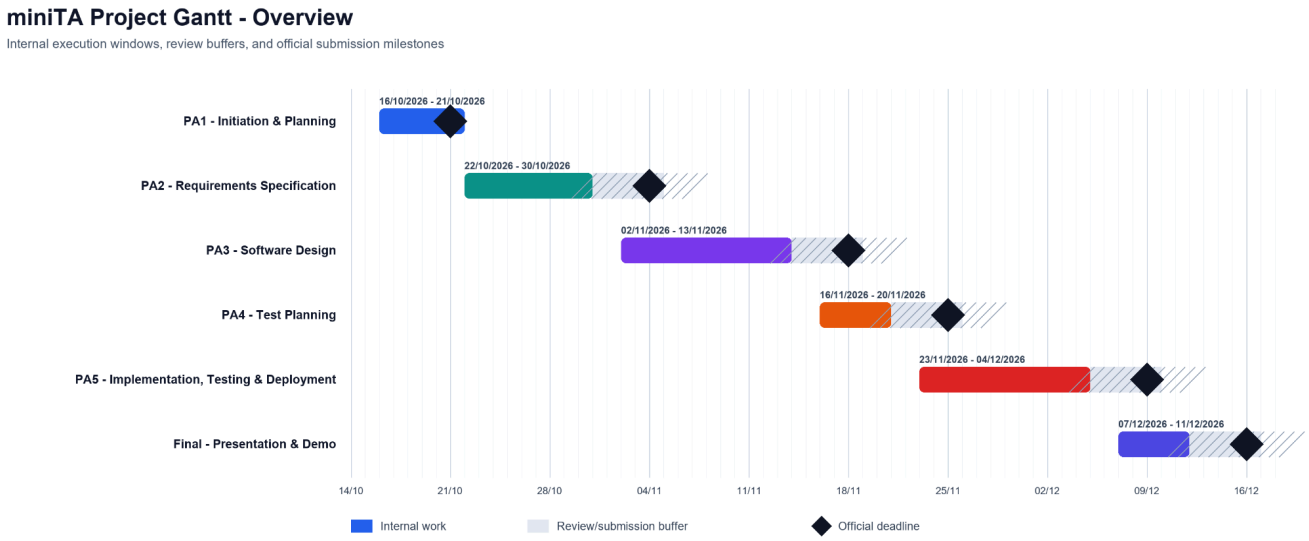
WBS ID	Work package	Owner	Effort (person-hours)	Method / assumptions
1.1.1	Write Preliminary Problem Statement & Product Position	Mẫn	3	Expert Judgement
1.1.2	Define Stakeholders, Users, and MoSCoW Feature List	Mẫn	2	Expert Judgement
1.1.3	Define Out-of-scope list and product boundaries	Mẫn	2	Expert Judgement
1.2.1	Draw C4 System Context diagram	Vân Anh	4	Expert Judgement
1.2.2	Define Technology Stack and Operating Environment	Vân Anh	2	Expert Judgement
1.2.3	Define Branching, PR, CI conventions and Repository Structure	Vân Anh	2	Analogy (from previous projects)
1.3.1	Build Work Breakdown Structure (Levels 1-3)	Nghi	5	Three-point estimate (O:3, M:5, P:7)
1.3.2	Synthesize effort estimates from all members	Nghi	2	Expert Judgement
1.3.3	Create Risk Register	Nghi	3	Three-point estimate (O:3, M:5, P:7)
1.4.1	Create Schedule	Triết	4	Expert Judgement
1.4.2	Define Roles, Backups, and Cost Estimate	Triết	2	Expert Judgement
1.4.3	Consolidate PA1 document	Triết	3	Expert Judgement
2.1.1	Elicit core recruitment business rules	Mẫn	4	Expert Judgement
2.1.2	Document data privacy & compliance rules	Mẫn	2	Expert Judgement
2.2.1	Draft Functional Requirements (FR) list with IDs	Triết	6	Three-point estimate
2.2.2	Draft Non-functional Requirements (NFR) list	Triết	3	Expert Judgement
2.3.1	Draw Use Case Diagram encompassing all Actors	Mẫn	4	Expert Judgement

2.3.2	Write Use Case Specifications for Applicant workflows	Mẫn	6	Three-point estimate
2.3.3	Write Use Case Specifications for HR workflows	Mẫn	6	Three-point estimate
2.4.1	Create low-fidelity mockups for Application Form	Vân Anh	4	Expert Judgement
2.4.2	Create low-fidelity mockups for Status Tracking	Vân Anh	3	Expert Judgement
2.5.1	Create low-fidelity mockups for Job Posting & Rubric Config	Vân Anh	5	Expert Judgement
2.5.2	Create low-fidelity mockups for Applicant Inbox & Dashboard	Vân Anh	5	Expert Judgement
2.6.1	Map FRs to Use Cases and UI Mockups	Nghi	4	Expert Judgement
3.1.1	Draw C4 Container Diagram	Vân Anh	5	Three-point estimate
3.1.2	Draw C4 Component Diagram for core modules	Vân Anh	6	Three-point estimate
3.2.1	Document ADR for Backend Framework selection	Triết	3	Expert Judgement
3.2.2	Document ADR for Database selection	Triết	3	Expert Judgement
3.3.1	Identify core entities and relationships	Nghi	4	Expert Judgement
3.3.2	Draw formal ERD	Nghi	4	Three-point estimate
3.4.1	Write detailed table schemas and data types	Nghi	5	Expert Judgement
3.4.2	Define primary keys, foreign keys, and indexes	Nghi	3	Expert Judgement
3.5.1	Draw Class Diagrams for core logic	Triết	8	Three-point estimate
3.5.2	Draw Sequence Diagrams for critical use cases	Triết	7	Three-point estimate
3.6.1	Create high-fidelity UI designs	Vân Anh	10	Three-point estimate

3.6.2	Map complete screen navigation flows	Vân Anh	4	Expert Judgement
4.1.1	Define Test Scope, Levels, and Types	Mẫn	3	Expert Judgement
4.1.2	Define Entry and Exit Criteria	Mẫn	2	Expert Judgement
4.2.1	Specify testing environments (local/cloud)	Mẫn	2	Expert Judgement
4.2.2	Define defect logging and tracking workflow	Mẫn	2	Analogy (from typical workflows)
4.3.1	Identify input fields for BVA & EP analysis	Nghi	4	Expert Judgement
4.3.2	Document resulting test data sets	Nghi	3	Expert Judgement
4.4.1	Write positive/negative test cases for CV upload	Nghi	4	Three-point estimate
4.4.2	Write test cases for Magic-link generation	Nghi	3	Three-point estimate
4.5.1	Write test cases for Job Posting & Rubric setup	Nghi	4	Expert Judgement
4.5.2	Write test cases for Candidate matching and scoring	Nghi	5	Three-point estimate
5.1.1	Implement Auth APIs & Database integration	Nghi	8	Three-point estimate
5.1.2	Build Login & Account UI components	Vân Anh	6	Expert Judgement
5.2.1	Implement Job CRUD APIs	Triết	10	Three-point estimate
5.2.2	Build Job Management UI components	Vân Anh	8	Expert Judgement
5.3.1	Integrate LLM API & JSON Schema validation	Triết	12	Three-point estimate
5.3.2	Build Rubric Configuration UI	Vân Anh	6	Expert Judgement
5.4.1	Implement Application submission APIs & File Upload	Nghi	10	Three-point estimate
5.4.2	Build Public Application Form UI	Vân Anh	6	Expert Judgement

5.5.1	Implement Application submission APIs & File Upload	Nghi	10	Three-point estimate (Risk R7)
5.5.2	Build Public Application Form UI	Vân Anh	5	Three-point estimate
5.6.1	Implement Email Service & Magic-link logic	Nghi	12	Three-point estimate (Risk R4)
5.6.2	Implement Rule-based text matching engine	Triết	14	Three-point estimate
5.7.1	Implement candidate filtering & human scorecard APIs	Triết	8	Expert Judgement
5.7.2	Build Applicant Inbox UI	Vân Anh	6	Expert Judgement
5.8.1	Implement statistics aggregation APIs	Triết	6	Expert Judgement
5.8.2	Build Dashboard Charts UI	Vân Anh	6	Analogy (Reusing chart libraries)
5.9.1	Write automated Unit Tests (Pytest/Vitest)	Mẫn	12	Three-point estimate
5.9.2	Execute System Testing & Log defects	Mẫn	15	Three-point estimate
5.9.3	Configure GitHub Actions CI workflow	Nghi	5	Expert Judgement
5.9.4	Deploy Database & App Containers to Cloud	Triết	8	Three-point estimate
6.1.1	Draft presentation outline and content	Mẫn	4	Expert Judgement
6.1.2	Design and format presentation slides	Mẫn	5	Expert Judgement
6.2.1	Write step-by-step demo script	Vân Anh	3	Expert Judgement
6.2.2	Record and edit fallback demo video	Vân Anh	6	Three-point estimate
6.3.1	Conduct internal team rehearsal	Triết	3	Expert Judgement
6.3.2	Refine timing and transitions	Triết	2	Expert Judgement

4.4 Schedule



The Gantt chart distinguishes internal work completion from the official PA deadline. The days between them are review and submission buffers. Preparatory work for a later phase may begin during a review buffer, but no downstream deliverable may treat the previous phase as baselined until its official phase gate is passed.

ID	Phase	Internal start	Internal finish	Official milestone	Depends on
S1	PA1 – Initiation and planning	16/10/2026	21/10/2026	PA1 baseline – 21/10/2026	None
S2	PA2 – Requirements	22/10/2026	30/10/2026	Requirements baseline – 04/11/2026	S1
S3	PA3 – Design	02/11/2026	13/11/2026	Design baseline – 18/11/2026	S2 internal completion; S2 gate before baselining

S4	PA4 – Test planning	16/11/2026	20/11/2026	Test-plan baseline – 25/11/2026	S3 internal completion; S3 gate before baselining
S5	PA5 – Implementation, testing and deployment	23/11/2026	04/12/2026	Release baseline – 09/12/2026	S4 internal completion; S4 gate before baselining
S6	Final presentation and demo	07/12/2026	11/12/2026	Final demonstration – 16/12/2026	S5 internal completion; S5 gate before final delivery

The phase-gate critical path is **S1** → **S2** → **S3** → **S4** → **S5** → **S6**. A delayed phase gate delays every downstream baseline. Effort in Section 4.3 is measured in person-hours; the calendar dates above represent duration and review buffers, so the two values are not interchangeable.

4.5 Roles and Responsibilities

Role	Main	Backup	Responsibilities
Team Lead / Project Manager	Nguyễn Cao Triết	Phùng Gia Mẫn	Plan phases, assign and track work, maintain reports, integrate PA documents and act as contact point.
Business Analyst	Phùng Gia Mẫn	Nguyễn Cao Triết	Elicit and document requirements, control scope, maintain requirement IDs and organise reviews.
Designer / Developer	Tống Vân Anh	Nguyễn Cao Triết	Design architecture, database, APIs and UI; maintain C4/UML models and technical decisions.
Tester	Gia Mẫn; Thục Nghi	Tống Vân Anh	Prepare test plans and cases, manage test data, execute tests and report defects and results.

4.6 Human Resources and Cost Estimate

The labour estimate uses the bottom-up effort from every Level-3 WBS work package. A rate of **100,000 VND per person-hour** represents the planning value of junior software-engineering work; it is not a salary paid to team members. The 376-hour plan averages 94 hours per member, or about 9.4 hours per member per week over ten project weeks.

Assumption	Value
Junior rate (VND/person-hour)	100,000
Planned total effort (person-hours)	376

Phase	Effort (hours)	Rate (VND/hour)	Labour cost (VND)
PA1 – Initiation and planning	34	100,000	3,400,000
PA2 – Requirements	52	100,000	5,200,000
PA3 – Design	62	100,000	6,200,000
PA4 – Test planning	32	100,000	3,200,000
PA5 – Implementation, testing and deployment	173	100,000	17,300,000
Final presentation and demo	23	100,000	2,300,000
Total labour	**376**		**37,600,000**

Member	Assigned tasks	Estimated effort (hours)	Effort share	Rate (VND/hour)	Estimated labour compensation (VND)
Phùng Gia Mẫn	16	74	19.7%	100,000	7,400,000
Nguyễn Thục Nghi	19	98	26.1%	100,000	9,800,000
Nguyễn Cao Triết	17	102	27.1%	100,000	10,200,000
Tống Vân Anh	20	102	27.1%	100,000	10,200,000
Total	**72**	**376**	**100%**		**37,600,000**

Other cost	Assumption	Estimated cost (VND)
GitHub, Jira and Discord	Education/free tiers	0
Hosting, PostgreSQL, file storage and email	Free tiers are preferred; a capped deployment budget is reserved for the final demo	500,000
LLM API usage	Capped development and demo budget	500,000
Contingency	Unexpected hosting, storage, email or API usage	500,000
Total other costs		**1,500,000**
Total estimated project cost	**Labour value plus expected cash costs**	**39,100,000**

The expected direct cash budget is 1,500,000 VND. Free tiers may reduce the actual cash spend, but the budget retains a deployment allowance so the final demo does not depend entirely on free-tier availability.

5 Risk Management

ID	Risk	Type	Probability	Loss	RE	Mitigation / contingency	Owner
R1	Key member unavailable during PA5	People	2	5	10	**Mitigation:** Mandate pull request reviews for knowledge sharing. Assign Backup Roles from the start. **Contingency:** Reallocate tasks; backup roles (e.g., BA or Designer) take over incomplete development/testing tasks.	Triết
R2	Scope growth or changing requirements	Requirements	3	4	12	**Mitigation:** PM and BA strictly enforce MoSCoW prioritization, focusing only on the 8 Must-have features. **Contingency:** PM makes a final decision to immediately drop any out-of-scope features to ensure progress.	Triết
R3	Incorrect AI-generated rubric	Technology	3	3	9	**Mitigation:** Implement strict JSON schema validation for the LLM API output. **Contingency:** Require a mandatory manual review and approval step by the Recruiter; allow manual rubric entry if AI fails.	Mẫn
R4	Failed CV text extraction	Technology	3	4	12	**Mitigation:** Restrict the system to support only selectable-text PDFs, completely excluding OCR for scanned images. **Contingency:** System automatically flags unreadable CVs and forwards the original PDF for manual review by HR.	Nghi

R5	Underestimated implementation effort	Estimation	3	4	12	**Mitigation:** Break down WBS into granular, manageable micro-tasks and apply the Three-point estimating technique. **Contingency:** Simplify UI/UX complexities and temporarily reallocate testing personnel to assist with backend/frontend development.	Mẫn
R6	CV or personal-data exposure	Security	2	5	10	**Mitigation:** Never commit secrets to Git. Use only fictional or consented test data. Enforce strict Database and File Storage authorization. **Contingency:** Immediately disable the affected cloud file access, revoke access tokens, and scrub the database.	Nghi
R7	Email or magic-link integration failure	Technology	2	4	8	**Mitigation:** Develop and execute Unit Tests for the Email Service early in the development stages. **Contingency:** Display the generated magic-link URL directly on the internal Recruiter dashboard so it can be copied and sent manually.	Nghi
R8	Task blockages due to mismanaged dependencies between members	Organisational	3	4	12	**Mitigation:** Clearly map out all task dependencies in the Gantt chart during the planning phase. Enforce the team contract's 24-hour response rule on Discord to communicate and resolve blockers quickly. **Contingency:** If a dependency remains blocked, PM temporarily reassigns the stalled member to independent tasks until the blocking task is completed.	Triết

6 Tools & Collaboration Setup

Purpose	Tool and working rule
Course announcements and PA submission	Moodle
Team communication	A private Discord team channel with the TA included. Members must acknowledge questions and blockers within 24 hours.
Task tracking	Jira project SCRUM. Every Level-3 WBS work package has an owner, due date, status, and acceptance evidence. Contribution evidence links to SCRUM-6 through SCRUM-9 are listed in Section 1.
Source control and review	GitHub repository accessible to the instructor and TA. The main branch is protected, direct pushes are disabled, and changes are merged through reviewed pull requests.
Weekly management	A 15-minute status meeting and a weekly report are completed every week from week 2 through week 10. Reports are committed to /docs/management/weekly by **Wednesday at 23:59 (Asia/Saigon)**.

Repository structure:

Folder	Content
/src	Source code
/docs/management	Project plan; weekly status reports in /docs/management/weekly
/docs/requirements	Vision, SRS, use cases, mockups
/docs/design	Architecture (C4), ADRs, UML models, UI design
/docs/test	Test plan, test cases, test reports
/docs/ai-usage	AI usage logs
/pa/PA1 ... /pa/PA5	One sub-folder per PA submission (PDF + source files)

Repository structure proposal:

The structure above is kept as is. Inside /src, the code is split into /src/frontend (React + TypeScript application) and /src/backend (FastAPI application organised into modules). The repository root contains README.md, .gitignore, .env.example, docker-compose.yml, and a .github folder holding the CI workflow and the pull request template. Placeholder .gitkeep files are removed once a folder has real content.

Branch, pull-request and CI conventions:

Branches: main is protected, and direct pushes are disabled. Work is done on short-lived branches named <type>/<short-description>, for example feat/job-posting-form, fix/pdf-extraction-error or docs/c4-system-context. Each branch covers one task and is deleted after merging.

Commits: Conventional Commits (feat:, fix:, docs:, test:), for example feat: add job posting form.

Pull requests: every pull request links to its task on the board and describes what changed and how it was tested. It needs approval from at least one teammate other than the author, and all review comments must be resolved before merging. Pull requests are merged with squash merge to keep the history readable.

CI: a GitHub Actions workflow runs on every pull request. It installs dependencies, runs linting (Ruff for Python, ESLint for TypeScript), runs the back-end tests (pytest), and runs the front-end tests and build (Vitest). A pull request can be merged only when all checks pass.

Secrets and data: .env files are listed in .gitignore, and only .env.example with placeholder values is committed. Only fictional or consented test data is used, and CV files are never committed to the repository.

7 AI Usage Log

#	Member	Tool	Task / purpose	How the output was verified and changed
1	Nguyễn Cao Triết	OpenAI Codex	Reviewed the PA1–PA5 templates and drafted the PA1 planning content, including phase gates, the schedule, roles and backup roles, contribution evidence, and the preliminary cost estimate.	Checked the output against the official course deadlines, the agreed team roles, and the PA1 template. Kept each member's contribution at 25%, verified the total was 100%, separated working-day duration from person-hour effort, and marked the cost estimate as provisional because bottom-up effort estimates were not available.
2	Nguyễn Cao Triết	OpenAI Codex	Compared miniTA's proposed recruitment features with SmartRecruiters concepts and refined the product boundary, AI-assisted rubric generation, resume extraction, candidate matching, and application-status workflow.	Retained only features suitable for the course project. Excluded a recruitment social network and full enterprise ATS scope, kept human review as the final decision point, and required transparent criteria instead of an unexplained AI score.
3	Phùng Gia Mẫn	Gemini	Drafted Section 2.1 (Preliminary Problem Statement) and improved the English wording of the PA0 requirements used in Sections 2.2, 2.3, and the explicit out-of-scope list.	Compared the draft with the team's PA0 content and Jira acceptance criteria. Revised wording that could change business meaning and kept the agreed product boundaries and recruitment workflow unchanged.
4	Tống Vân Anh	Claude	Reviewed the completed PA1 sections against the	Added the missing contribution and AI-usage evidence, applied wording

			task acceptance criteria and identified missing contribution evidence, the AI usage log, and minor wording issues.	corrections that matched the team decisions, and left unresolved technical choices open instead of accepting unapproved AI suggestions.
5	Tống Vân Anh	Claude	Explained technical terms used in Sections 3.1–3.3 and 6, including modular monolith, CI, squash merge, and metadata.	Used the explanations for learning only. Did not copy additional explanatory text into the PA1 document.
6	Tống Vân Anh	Claude	Drafted PA1 Sections 3.1–3.3 and the technical conventions in Section 6, including the technology stack, operating environment, repository structure, branch, pull-request, and CI conventions.	Checked the draft against the PA1 template and task acceptance criteria. Removed the extra `External services` row, retained React with TypeScript, FastAPI, PostgreSQL, and a modular monolith, and marked unresolved hosting, storage, merge, and repository-layout choices for team confirmation.
7	Tống Vân Anh	Claude	Explained the C4 System Context diagram, proposed people, external systems, and relationship labels for miniTA, and provided draw.io drawing and styling guidance	Drew and edited the diagram manually in draw.io. Shortened repeated relationship labels and kept the diagram at System Context level without adding React, FastAPI, or PostgreSQL as context-level boxes.
8	Nguyễn Thục Nghi	Gemini	Drafted Section 4.2 (Work Breakdown Structure), decomposed the planned feature set into a three-level hierarchy, and mapped the work packages to the Excel Gantt chart.	Checked that the planned features were represented, every Level-3 work package had one owner, and the dates matched the course timeline. Kept F08 Recruitment Dashboard provisional because the team had not confirmed it as a required Must-have feature.